

04-jj-dbscan-tsne-clientes

May 14, 2026

1 Proyecto 3 — DBSCAN, t-SNE y Comparación de Modelos de Clustering

1.1 Integrantes

- Daniel Fernando Salgado Santamaría
- Jairo Wladimir Jhayya Perlaza
- Luis Gabriel Salgado Santamaría
- Oscar Paul Naranjo Castro

Fecha: 2026-05-14

Notebook: 04-jj-dbscan-tsne-clientes.ipynb

1.2 Objetivo

Implementar DBSCAN como segundo modelo de clustering no supervisado y aplicar t-SNE para visualización bidimensional de agrupamientos complejos. Se comparan los resultados de DBSCAN contra K-Means (notebook 03) en términos de perfiles de cluster, distribución de puntos y detección de ruido. Finalmente se responden las preguntas de reflexión del docente.

1.3 Configuración del entorno

Se importan todas las librerías necesarias para clustering (DBSCAN), reducción de dimensionalidad (PCA, t-SNE), visualización y análisis estadístico.

```
# Importa librerías estándar de manejo de rutas y sistema.
import sys
from pathlib import Path

# Importa librerías numéricas y de análisis de datos.
import numpy as np
import pandas as pd

# Importa herramientas de visualización de alta calidad.
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns

# Importa modelos y métricas de scikit-learn para clustering y reducción.
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN, KMeans
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors

# Verifica que el notebook esté usando el entorno virtual correcto.
print("Python ejecutándose desde:")
print(sys.executable)

```

Python ejecutándose desde:

g:\ProyectosPython\proyecto3\.venv\Scripts\python.exe

1.4 Configuración visual global

Se establece el estilo, resolución y tamaño de fuente para todos los gráficos del notebook, asegurando uniformidad visual y calidad de exportación a 400 DPI.

```

# Configura el estilo visual global: grilla limpia, contexto 'talk' para
↪ fuentes legibles.
sns.set_theme(style="whitegrid", context="talk")

# Establece resolución de pantalla y de guardado para gráficos de alta calidad.
plt.rcParams["figure.dpi"]      = 140
plt.rcParams["savefig.dpi"]     = 400
plt.rcParams["axes.titlesize"]  = 16
plt.rcParams["axes.labelsize"]  = 13
plt.rcParams["xtick.labelsize"] = 11
plt.rcParams["ytick.labelsize"] = 11
plt.rcParams["legend.fontsize"] = 11
plt.rcParams["font.weight"]     = "regular"
plt.rcParams["axes.titleweight"] = "bold"
plt.rcParams["axes.labelweight"] = "bold"

# Define la paleta de colores vivos y profesionales usada en todo el proyecto.
PALETTE_VIVID = [
    "#0F766E", # teal intenso
    "#2563EB", # azul vivo
    "#DC2626", # rojo elegante
    "#F59E0B", # ámbar
    "#7C3AED", # violeta controlado
    "#059669", # verde fuerte
    "#EA580C", # naranja profundo
    "#DB2777", # magenta elegante
    "#0891B2", # cyan fuerte
    "#65A30D", # lima oscura
]

```

```
print("Configuración visual aplicada correctamente.")
```

Configuración visual aplicada correctamente.

1.5 Definición de rutas del proyecto

Se definen rutas absolutas de forma reproducible para que el notebook funcione correctamente desde cualquier máquina del equipo sin necesidad de modificar paths manualmente.

```
# Define la raíz del proyecto de forma dinámica según desde dónde se ejecute el  
↪ notebook.  
PROJECT_ROOT = (  
    Path.cwd().resolve().parent  
    if Path.cwd().name == "notebooks"  
    else Path.cwd().resolve()  
)  
  
# Define rutas de datos procesados y carpetas de figuras y tablas.  
PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"  
FIGURES_DBSCAN = PROJECT_ROOT / "reports" / "figures" / "dbscan"  
FIGURES_TSNE = PROJECT_ROOT / "reports" / "figures" / "tsne"  
FIGURES_CMP = PROJECT_ROOT / "reports" / "figures" / "clustering_clientes"  
TABLES_DIR = PROJECT_ROOT / "reports" / "tables"  
  
# Crea las carpetas de destino si no existen.  
for path in [FIGURES_DBSCAN, FIGURES_TSNE, FIGURES_CMP, TABLES_DIR]:  
    path.mkdir(parents=True, exist_ok=True)  
  
print("PROJECT_ROOT :", PROJECT_ROOT)  
print("PROCESSED_DIR :", PROCESSED_DIR)  
print("FIGURES_DBSCAN:", FIGURES_DBSCAN)  
print("FIGURES_TSNE :", FIGURES_TSNE)
```

```
PROJECT_ROOT : G:\ProyectosPython\proyecto3\desarrollo3  
PROCESSED_DIR : G:\ProyectosPython\proyecto3\desarrollo3\data\processed  
FIGURES_DBSCAN: G:\ProyectosPython\proyecto3\desarrollo3\reports\figures\dbscan  
FIGURES_TSNE : G:\ProyectosPython\proyecto3\desarrollo3\reports\figures\tsne
```

1.6 Funciones auxiliares

Se definen funciones reutilizables para guardar figuras y anotar barras con valores, garantizando uniformidad visual en todos los gráficos del notebook.

```
# Define función para guardar la figura actual en alta resolución en la carpeta  
↪ indicada.  
def f_save_plot(filename, folder):  
    """Guarda la figura activa en la carpeta especificada a 400 DPI."""
```

```

plt.savefig(folder / filename, dpi=400, bbox_inches="tight",
↪facecolor="white")

# Define función para anotar barras verticales con su valor encima de cada
↪barra.
def f_annotate_bars(ax, fmt="{:.0f}", suffix="", fontsize=10, weight="bold"):
    """Añade etiquetas sobre cada barra vertical del gráfico."""
    for patch in ax.patches:
        height = patch.get_height()
        if pd.notnull(height):
            x = patch.get_x() + patch.get_width() / 2
            ax.annotate(
                fmt.format(height) + suffix,
                (x, height),
                ha="center", va="bottom",
                fontsize=fontsize, fontweight=weight,
                xytext=(0, 6), textcoords="offset points"
            )

# Define función para anotar barras horizontales con su valor al final de cada
↪barra.
def f_annotate_bars_horizontal(ax, fmt="{:.2f}", suffix="", fontsize=10,
↪weight="bold"):
    """Añade etiquetas al final de cada barra horizontal del gráfico."""
    for patch in ax.patches:
        width = patch.get_width()
        y = patch.get_y() + patch.get_height() / 2
        if pd.notnull(width):
            ax.annotate(
                fmt.format(width) + suffix,
                (width, y),
                ha="left", va="center",
                fontsize=fontsize, fontweight=weight,
                xytext=(6, 0), textcoords="offset points"
            )

print("Funciones auxiliares definidas correctamente.")

```

Funciones auxiliares definidas correctamente.

1.7 1. Carga y verificación de datos

Se carga el archivo `customer_features.csv` generado en el notebook 02 (EDA), que contiene las métricas consolidadas por cliente: edad, número de sesiones, órdenes, ingresos, ticket promedio y reseñas. Este archivo es la base para todos los modelos de este notebook.

```
# Carga el archivo de características de clientes generado en el notebook de
↳ EDA.
customer_features = pd.read_csv(PROCESSED_DIR / "customer_features.csv")

# Muestra las dimensiones y primeras filas para verificar integridad del
↳ archivo.
print("Dimensiones:", customer_features.shape)
customer_features.head()
```

Dimensiones: (20000, 10)

	customer_id	country	age	signup_date	marketing_opt_in	n_sessions	\
0	1	JP	71	2020-09-04	True	5.0	
1	2	IN	26	2020-04-05	False	3.0	
2	3	BR	21	2023-08-31	True	5.0	
3	4	FR	63	2022-06-30	True	9.0	
4	5	BR	19	2022-07-22	True	9.0	

	n_orders	gross_revenue_usd	avg_order_value_usd	n_reviews
0	2.0	115.39	57.695	1.0
1	2.0	68.52	34.260	0.0
2	1.0	66.72	66.720	0.0
3	2.0	279.86	139.930	0.0
4	3.0	271.29	90.430	4.0

```
# Verifica que no haya valores nulos en ninguna de las columnas del dataset.
print("Valores nulos por columna:")
print(customer_features.isnull().sum())
```

Valores nulos por columna:

customer_id	0
country	0
age	0
signup_date	0
marketing_opt_in	0
n_sessions	0
n_orders	0
gross_revenue_usd	0
avg_order_value_usd	0
n_reviews	0

dtype: int64

1.8 2. Preprocesamiento para clustering

DBSCAN y t-SNE son sensibles a la escala de las variables. Se seleccionan únicamente las variables numéricas de comportamiento y se aplica `StandardScaler` para normalizar cada variable a media 0 y desviación estándar 1, eliminando el efecto de magnitud.

```

# Define las variables numéricas de comportamiento para el análisis de
↳clustering.
# Se excluyen identificadores (customer_id) y fechas (signup_date).
FEATURE_COLS = [
    "age",
    "n_sessions",
    "n_orders",
    "gross_revenue_usd",
    "avg_order_value_usd",
    "n_reviews",
]

# Filtra solo las columnas disponibles en el dataset para evitar errores.
FEATURE_COLS = [c for c in FEATURE_COLS if c in customer_features.columns]
print("Variables utilizadas para clustering:")
print(FEATURE_COLS)

# Extrae la matriz de características y escala a media=0, std=1.
X = customer_features[FEATURE_COLS].copy()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"\nMatriz escalada: {X_scaled.shape[0]} clientes × {X_scaled.shape[1]}
↳variables")

```

Variables utilizadas para clustering:

```
['age', 'n_sessions', 'n_orders', 'gross_revenue_usd', 'avg_order_value_usd',
'n_reviews']
```

Matriz escalada: 20000 clientes × 6 variables

1.9 3. Búsqueda del parámetro óptimo para DBSCAN

Para determinar el valor óptimo de **eps** (radio de vecindad en DBSCAN) se usa el **K-Distance Graph**: se calcula la distancia al k-ésimo vecino más cercano de cada punto (donde $k = \text{min_samples} - 1$) y se grafica en orden descendente. El punto de mayor curvatura (*codo*) indica el valor de **eps** donde la densidad cambia abruptamente, separando clusters de ruido.

```

# Calcula las distancias al 4° vecino más cercano de cada punto (k=4 +
↳min_samples=5).
k = 4
nbrs = NearestNeighbors(n_neighbors=k).fit(X_scaled)
distances, _ = nbrs.kneighbors(X_scaled)

# Extrae la distancia al vecino más lejano (k-ésimo) y ordena de mayor a menor.
k_distances = np.sort(distances[:, -1])[:, -1]

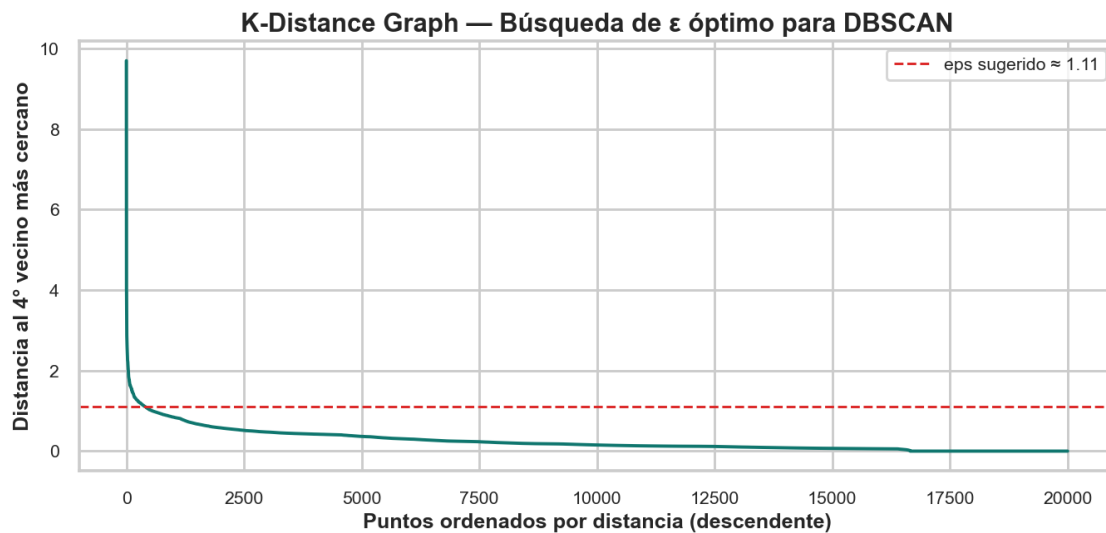
```

```

# Grafica el K-Distance Graph para identificar visualmente el codo (eps óptimo).
plt.figure(figsize=(10, 5))
plt.plot(k_distances, color=PALETTE_VIVID[0], linewidth=2)
plt.axhline(
    y=k_distances[int(len(k_distances) * 0.02)],
    color=PALETTE_VIVID[2], linestyle="--", linewidth=1.5,
    label=f"eps sugerido   {k_distances[int(len(k_distances)*0.02)]:.2f}"
)
plt.title("K-Distance Graph - Búsqueda de  óptimo para DBSCAN")
plt.xlabel("Puntos ordenados por distancia (descendente)")
plt.ylabel(f"Distancia al {k}º vecino más cercano")
plt.legend(fontsize=11)
plt.tight_layout()
f_save_plot("kdistance_graph_dbscan.png", FIGURES_DBSCAN)
plt.show()

# Imprime el valor sugerido de eps para guiar la selección del parámetro.
eps_sugerido = round(k_distances[int(len(k_distances) * 0.02)], 2)
print(f"Valor de eps sugerido por K-Distance Graph: {eps_sugerido}")

```



Valor de eps sugerido por K-Distance Graph: 1.11

1.10 4. Modelo DBSCAN — Ajuste de parámetros

Con el valor de `eps` identificado en el K-Distance Graph, se entrena el modelo DBSCAN. El parámetro `min_samples` controla la densidad mínima para que un punto sea considerado núcleo; un valor de 5 es adecuado para datasets con más de 1000 observaciones.

Los puntos etiquetados como `-1` son considerados **ruido** (outliers que no pertenecen a ningún cluster), lo cual es una característica diferenciadora respecto a K-Means.

```

# Entrena el modelo DBSCAN con el eps identificado y min_samples=5.
EPS = eps_sugerido
MIN_SAMPLES = 5

dbscan = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES)
labels_dbscan = dbscan.fit_predict(X_scaled)

# Agrega las etiquetas DBSCAN al DataFrame original para análisis posterior.
customer_features["cluster_dbscan"] = labels_dbscan

# Calcula métricas clave del resultado del modelo.
n_clusters_dbscan = len(set(labels_dbscan)) - (1 if -1 in labels_dbscan else 0)
n_noise = (labels_dbscan == -1).sum()
pct_noise = n_noise / len(labels_dbscan) * 100

print(f"Parámetros usados → eps={EPS}, min_samples={MIN_SAMPLES}")
print(f"Clusters encontrados: {n_clusters_dbscan}")
print(f"Puntos de ruido : {n_noise} ({pct_noise:.1f}% del total)")
print("\nDistribución de clusters (incluyendo ruido=-1):")
print(customer_features["cluster_dbscan"].value_counts().sort_index())

```

Parámetros usados → eps=1.11, min_samples=5

Clusters encontrados: 5

Puntos de ruido : 266 (1.3% del total)

Distribución de clusters (incluyendo ruido=-1):

cluster_dbscan

-1 266

0 19710

1 9

2 5

3 5

4 5

Name: count, dtype: int64

```

# Calcula el Silhouette Score de DBSCAN excluyendo los puntos de ruido (-1).
# El silhouette solo es válido cuando hay al menos 2 clusters con puntos
↪no-ruido.
mask_no_noise = labels_dbscan != -1

if n_clusters_dbscan >= 2 and mask_no_noise.sum() > 1:
    sil_dbscan = silhouette_score(
        X_scaled[mask_no_noise], labels_dbscan[mask_no_noise]
    )
    print(f"Silhouette Score DBSCAN (sin ruido): {sil_dbscan:.4f}")
else:
    sil_dbscan = None

```

```
print("No se puede calcular Silhouette Score: clusters insuficientes.")
```

Silhouette Score DBSCAN (sin ruido): 0.4491

1.11 5. Visualización de resultados DBSCAN

Se grafican tres aspectos clave del modelo DBSCAN: 1. **Tamaño de cada cluster** (incluyendo ruido) con valores anotados en cada barra. 2. **Perfil promedio por cluster** en las variables de comportamiento. 3. **Scatter plot** de las dos variables más discriminantes coloreado por cluster DBSCAN.

```
# Gráfico 1: Tamaño de clusters DBSCAN incluyendo puntos de ruido
cluster_counts_dbscan = (
    customer_features["cluster_dbscan"]
    .value_counts()
    .sort_index()
    .reset_index()
)
cluster_counts_dbscan.columns = ["cluster", "count"]

# Asigna etiquetas descriptivas: ruido aparece como 'Ruido (-1)'.
cluster_counts_dbscan["label"] = cluster_counts_dbscan["cluster"].apply(
    lambda x: "Ruido (-1)" if x == -1 else f"Cluster {x}"
)

# Define colores: ruido en gris, clusters en paleta vívida.
colors_dbscan = []
vivid_idx = 0
for c in cluster_counts_dbscan["cluster"]:
    if c == -1:
        colors_dbscan.append("#9CA3AF")
    else:
        colors_dbscan.append(PALETTE_VIVID[vivid_idx % len(PALETTE_VIVID)])
        vivid_idx += 1

plt.figure(figsize=(max(8, len(cluster_counts_dbscan) * 1.5), 6))
ax = sns.barplot(
    data=cluster_counts_dbscan,
    x="label", y="count",
    palette=colors_dbscan,
    edgecolor="white", linewidth=0.8
)
f_annotateBars(ax, fmt="{:.0f}")
plt.title("Tamaño de clusters - DBSCAN (incluyendo ruido)")
plt.xlabel("Cluster")
plt.ylabel("Número de clientes")
plt.tight_layout()
f_save_plot("tamano_clusters_dbscan.png", FIGURES_DBSCAN)
```

```
plt.show()
print("Gráfico guardado: tamano_clusters_dbscan.png")
```

C:\Users\vboxuser\AppData\Local\Temp\ipykernel_7528\3575612432.py:26:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(
```

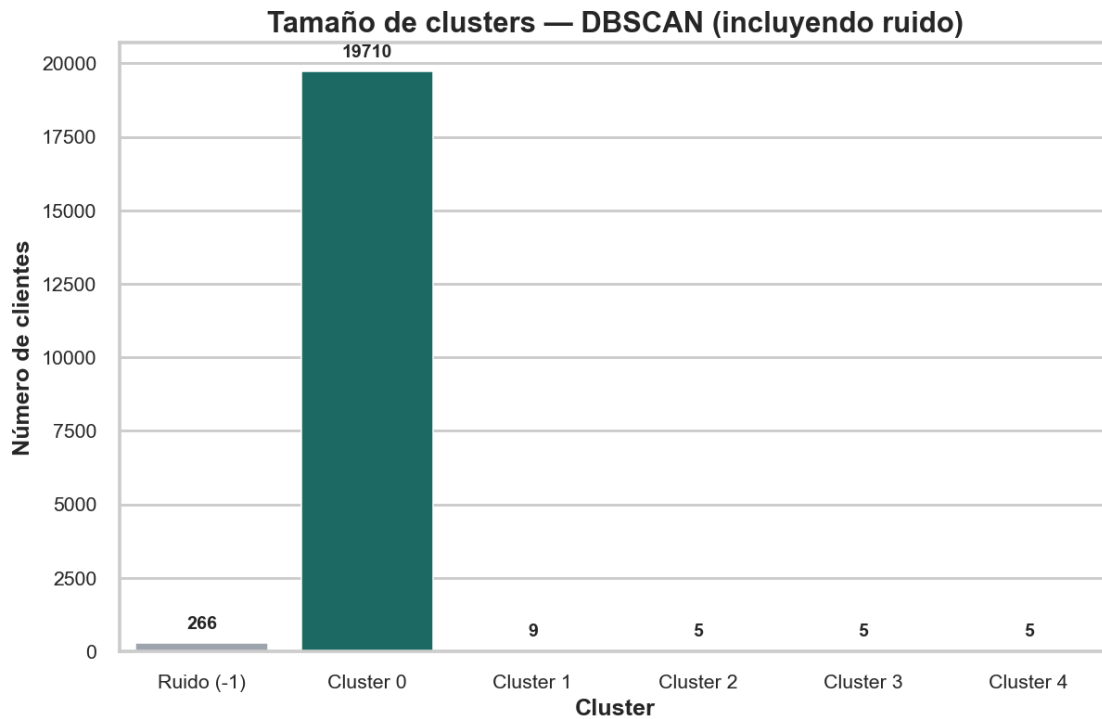


Gráfico guardado: tamano_clusters_dbscan.png

```
# Gráfico 2: Perfil promedio por cluster DBSCAN (excluye ruido)
# Calcula la media de cada variable de comportamiento agrupando por cluster_
↳ DBSCAN.
profile_dbscan = (
    customer_features[customer_features["cluster_dbscan"] != -1]
    .groupby("cluster_dbscan")[FEATURE_COLS]
    .mean()
    .reset_index()
)
```

```

# Convierte cluster_dbscan a string para usarlo como hue categórico en seaborn.
profile_dbscan["cluster_label"] = profile_dbscan["cluster_dbscan"].apply(
    lambda x: f"Cluster {int(x)}"
)

# Construye paleta como lista ordenada por aparición de clusters.
n_clusters_plot = profile_dbscan["cluster_label"].nunique()
palette_list = PALETTE_VIVID[:n_clusters_plot]

# Genera un subgráfico por variable para comparar el perfil de cada cluster.
n_vars = len(FEATURE_COLS)
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, var in enumerate(FEATURE_COLS):
    ax = axes[i]
    sns.barplot(
        data=profile_dbscan,
        x="cluster_label", y=var,
        hue="cluster_label",
        palette=palette_list,
        edgecolor="white", linewidth=0.8,
        legend=False,
        ax=ax
    )
    f_annotateBars(ax, fmt="{:.1f}")
    ax.set_title(f"Variable: {var}", fontweight="bold")
    ax.set_xlabel("Cluster DBSCAN")
    ax.set_ylabel("Media")

# Oculta el último subgráfico si hay espacio sobrante.
for j in range(n_vars, len(axes)):
    axes[j].set_visible(False)

plt.suptitle("Perfil promedio por cluster - DBSCAN",
            fontsize=18, fontweight="bold", y=1.01)
plt.tight_layout()
f_save_plot("perfil_promedio_clusters_dbscan.png", FIGURES_DBSCAN)
plt.show()
print("Gráfico guardado: perfil_promedio_clusters_dbscan.png")

```

Perfil promedio por cluster — DBSCAN

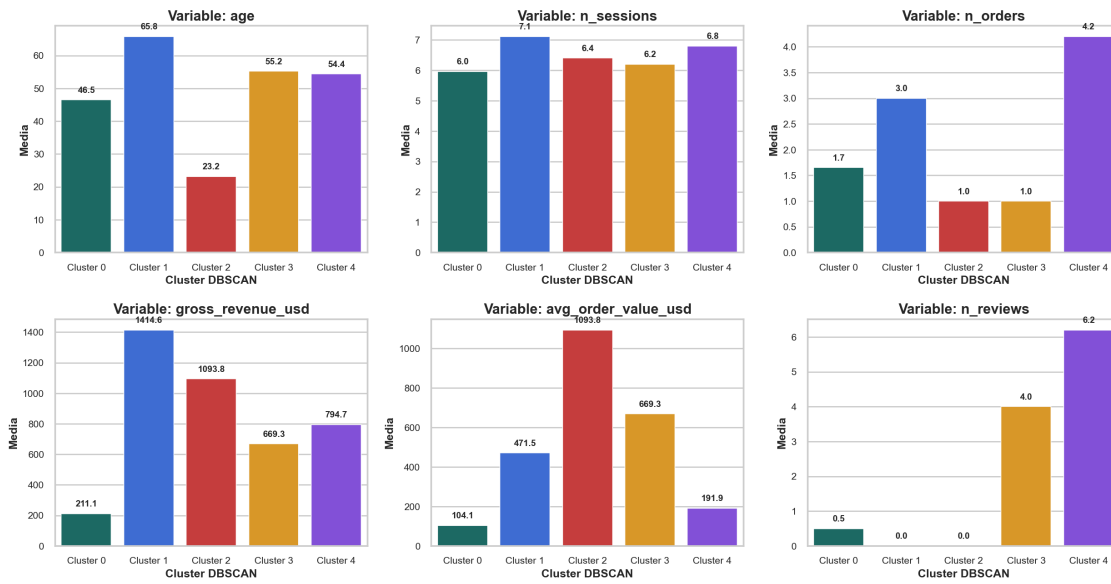


Gráfico guardado: perfil_promedio_clusters_dbscan.png

1.12 6. t-SNE — Reducción bidimensional

t-SNE (t-Distributed Stochastic Neighbor Embedding) es una técnica no lineal de reducción de dimensionalidad especialmente efectiva para visualizar estructuras de agrupamiento complejas en 2D. A diferencia de PCA (que preserva varianza global), t-SNE preserva relaciones de **vecindad local**, revelando agrupaciones que no son visibles en proyecciones lineales.

Parámetros clave: - **perplexity**: controla el balance entre estructura local y global (valor típico: 30–50). - **n_iter**: número de iteraciones para convergencia (mínimo recomendado: 1000). - **random_state**: fija la semilla para reproducibilidad.

```
# Aplica t-SNE sobre la matriz escalada para obtener representación 2D de los
↪ clientes.
# NOTA: En scikit-learn >= 1.5 el parámetro n_iter se renombró a max_iter.
tsne = TSNE(
    n_components=2,
    perplexity=40,
    max_iter=1000,
    random_state=42,
    learning_rate="auto",
    init="pca"          # inicialización con PCA mejora la estabilidad
)
X_tsne = tsne.fit_transform(X_scaled)
```

```

# Almacena las coordenadas t-SNE en el DataFrame para graficar con etiquetas de
↳ cluster.
customer_features["tsne_1"] = X_tsne[:, 0]
customer_features["tsne_2"] = X_tsne[:, 1]

print(f"t-SNE completado. Forma del resultado: {X_tsne.shape}")
print(f"Rango t-SNE 1: [{X_tsne[:,0].min():.2f}, {X_tsne[:,0].max():.2f}]")
print(f"Rango t-SNE 2: [{X_tsne[:,1].min():.2f}, {X_tsne[:,1].max():.2f}]")

```

t-SNE completado. Forma del resultado: (20000, 2)

Rango t-SNE 1: [-119.59, 97.59]

Rango t-SNE 2: [-109.01, 104.94]

1.13 8. Visualización t-SNE coloreada por clusters DBSCAN

Se usa la misma proyección t-SNE pero ahora coloreada con las etiquetas de **DBSCAN**. Los puntos de ruido (-1) se muestran en gris translúcido para no contaminar la lectura visual de los clusters reales.

```

# t-SNE coloreado por cluster DBSCAN
df_tsne_no_noise = customer_features[customer_features["cluster_dbscan"] != -1]
df_tsne_noise = customer_features[customer_features["cluster_dbscan"] == -1]

plt.figure(figsize=(11, 8))

# Grafica el ruido primero (fondo gris) para que los clusters queden encima.
plt.scatter(
    df_tsne_noise["tsne_1"], df_tsne_noise["tsne_2"],
    c="#D1D5DB", alpha=0.3, s=12,
    label=f"Ruido (-1) (n={len(df_tsne_noise):,})"
)

# Grafica cada cluster DBSCAN con color vívido.
for idx, cluster_id in enumerate(sorted(df_tsne_no_noise["cluster_dbscan"].
↳ unique())):
    subset = df_tsne_no_noise[df_tsne_no_noise["cluster_dbscan"] == cluster_id]
    plt.scatter(
        subset["tsne_1"], subset["tsne_2"],
        c=PALETTE_VIVID[idx % len(PALETTE_VIVID)],
        alpha=0.65, s=18,
        label=f"Cluster DBSCAN {cluster_id} (n={len(subset):,})"
    )

plt.title("t-SNE 2D - Coloreado por clusters DBSCAN",
        fontsize=16, fontweight="bold")
plt.xlabel("Dimensión t-SNE 1")
plt.ylabel("Dimensión t-SNE 2")
plt.legend(title="Cluster DBSCAN", fontsize=10, markerscale=2)

```

```
plt.tight_layout()
f_save_plot("tsne_clusters_dbscan.png", FIGURES_TSNE)
plt.show()
print("Gráfico guardado: tsne_clusters_dbscan.png")
```

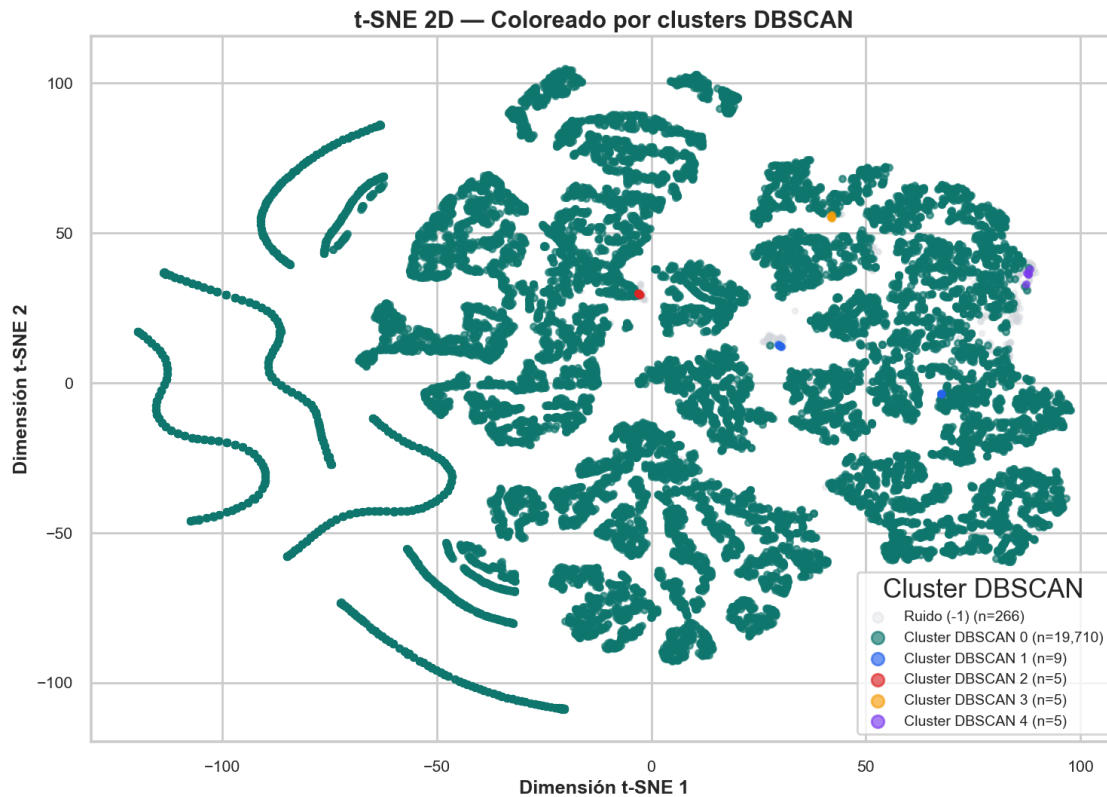


Gráfico guardado: tsne_clusters_dbscan.png

1.14 9. Comparación K-Means vs DBSCAN

Se construye una tabla y gráficos comparativos entre ambos modelos evaluando: - Número de clusters generados - Porcentaje de puntos clasificados como ruido (solo DBSCAN) - Silhouette Score de cada modelo - Capacidad de detectar formas arbitrarias vs esféricas

```
#
# CELDA DE INICIALIZACIÓN - Agrega columna "cluster_kmeans" a customer_features
#
# X_scaled ya existe en memoria (confirmado por diagnóstico).
# Solo se entrena K-Means y se añade la columna que faltaba.
#
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```

# Entrena K-Means con k=2 (óptimo del notebook 03, Silhouette=0.30)
kmeans_final = KMeans(n_clusters=2, random_state=42, n_init=20)
customer_features["cluster_kmeans"] = kmeans_final.fit_predict(X_scaled)

# Calcula silhouette score de K-Means (se usará en celda 33)
sil_kmeans = silhouette_score(X_scaled, customer_features["cluster_kmeans"])

# Recupera métricas de DBSCAN desde lo que ya existe en memoria
labels_db = customer_features["cluster_dbscan"]
n_noise = int((labels_db == -1).sum())
pct_noise = n_noise / len(labels_db) * 100
n_clusters_dbscan = int(labels_db[labels_db != -1].nunique())
mask_db = labels_db != -1
sil_dbscan = silhouette_score(X_scaled[mask_db], labels_db[mask_db])

# Verificación
print(f"cluster_kmeans agregado ")
print(f"Silhouette K-Means : {sil_kmeans:.4f}")
print(f"Silhouette DBSCAN : {sil_dbscan:.4f}")
print(f"Clusters DBSCAN : {n_clusters_dbscan} | Ruido: {n_noise}_
↳({pct_noise:.1f}%)")
print(f"\nDistribución K-Means:")
print(customer_features["cluster_kmeans"].value_counts().sort_index())

```

```

cluster_kmeans agregado
Silhouette K-Means : 0.2994
Silhouette DBSCAN : 0.4491
Clusters DBSCAN : 5 | Ruido: 266 (1.3%)

```

```

Distribución K-Means:
cluster_kmeans
0      7041
1     12959
Name: count, dtype: int64

```

```

# Tabla comparativa de métricas entre K-Means y DBSCAN
comparison_data = {
    "Modelo": ["K-Means", "DBSCAN"],
    "N° Clusters": [2,
↳n_clusters_dbscan],
    "Puntos de Ruido": ["0 (0.0%)", f"{n_noise}_
↳({pct_noise:.1f}%)"],
    "Silhouette Score": [f"{sil_kmeans:.4f}", f"{sil_dbscan:.
↳4f}"],
    "Detecta formas arbitrarias": ["No", "Sí"],
    "Sensible a outliers": ["Sí", "No (clasifica_
↳como ruido)"],

```

```

}

comparison_df = pd.DataFrame(comparison_data)
print("=" * 65)
print("TABLA COMPARATIVA: K-Means vs DBSCAN")
print("=" * 65)
print(comparison_df.to_string(index=False))

comparison_df.to_csv(TABLES_DIR / "comparacion_kmeans_dbscan.csv", index=False)
print("\nTabla guardada: comparacion_kmeans_dbscan.csv ")

```

```

=====
TABLA COMPARATIVA: K-Means vs DBSCAN
=====

```

Modelo	N° Clusters	Puntos de Ruido	Silhouette Score	Detecta formas arbitrarias Sensible a outliers
K-Means	2	0 (0.0%)	0.2994	No
DBSCAN	5	266 (1.3%)	0.4491	Sí

No (clasifica como ruido)

Tabla guardada: comparacion_kmeans_dbscan.csv

```

print("FIGURES_CMP:", FIGURES_CMP)
print("PALETTE_VIVID:", PALETTE_VIVID)

```

FIGURES_CMP:

G:\ProyectosPython\proyecto3\desarrollo3\reports\figures\clustering_clientes
 PALETTE_VIVID: ['#0F766E', '#2563EB', '#DC2626', '#F59E0B', '#7C3AED',
 '#059669', '#EA580C', '#DB2777', '#0891B2', '#65A30D']

```

# Comparación Silhouette Score: K-Means vs DBSCAN
import matplotlib.pyplot as plt

score_df = pd.DataFrame({
    "Modelo": ["K-Means", "DBSCAN"],
    "Silhouette Score": [sil_kmeans, sil_dbscan],
})

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(
    score_df["Modelo"],
    score_df["Silhouette Score"],
    color=[PALETTE_VIVID[0], PALETTE_VIVID[1]],
    edgecolor="white", linewidth=0.8, width=0.5
)

for bar in bars:

```

```

h = bar.get_height()
ax.annotate(f"{h:.4f}",
            xy=(bar.get_x() + bar.get_width() / 2, h),
            xytext=(0, 6), textcoords="offset points",
            ha="center", va="bottom", fontsize=11, fontweight="bold")

ax.set_title("Comparación Silhouette Score - K-Means vs DBSCAN",
            fontweight="bold")
ax.set_xlabel("Modelo de Clustering")
ax.set_ylabel("Silhouette Score")
ax.set_ylim(0, max(sil_kmeans, sil_dbscan) * 1.35)
plt.tight_layout()
plt.savefig(
    FIGURES_CMP / "comparacion_silhouette_kmeans_dbscan.png",
    dpi=400, bbox_inches="tight", facecolor="white"
)
plt.show()
print("Gráfico guardado ")

```

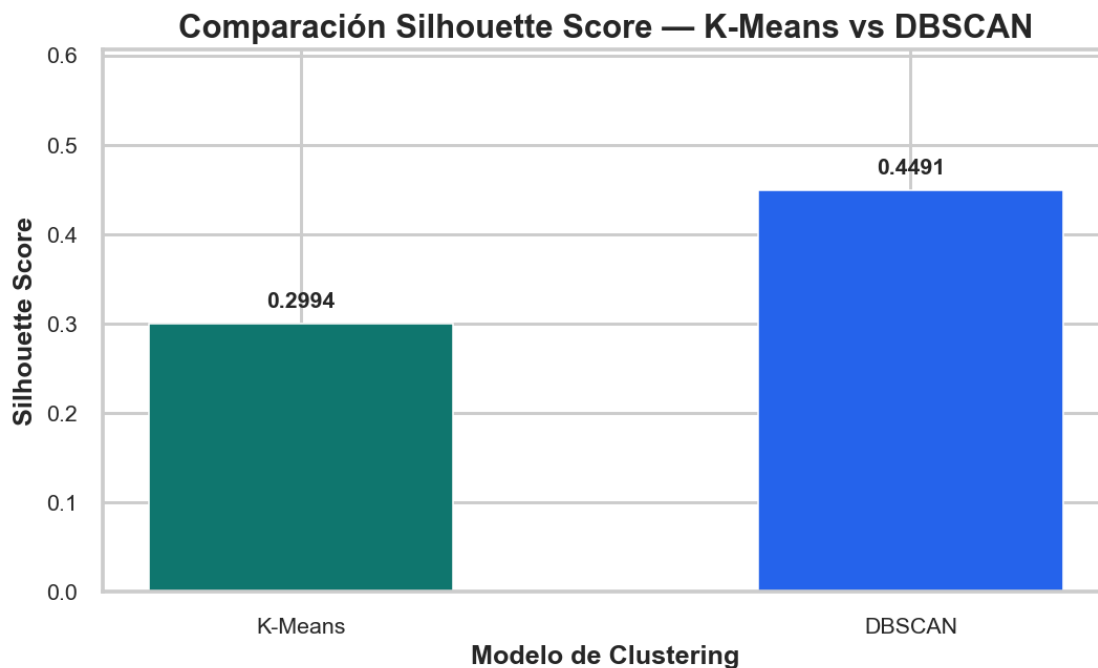


Gráfico guardado

```

# Distribución de clientes por cluster: K-Means vs DBSCAN
km_counts = (
    customer_features["cluster_kmeans"]
    .value_counts().sort_index().reset_index()
)

```

```

)
km_counts.columns = ["cluster", "count"]
km_counts["label"] = km_counts["cluster"].apply(lambda x: f"C{int(x)}")

db_counts = (
    customer_features["cluster_dbscan"]
    .value_counts().sort_index().reset_index()
)
db_counts.columns = ["cluster", "count"]
db_counts["label"] = db_counts["cluster"].apply(
    lambda x: "Ruido" if int(x) == -1 else f"C{int(x)}"
)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Panel izquierdo: K-Means
ax0 = axes[0]
km_colors = [PALETTE_VIVID[i % len(PALETTE_VIVID)] for i in
    range(len(km_counts))]
bars0 = ax0.bar(km_counts["label"], km_counts["count"],
    color=km_colors, edgecolor="white", linewidth=0.8)
for bar in bars0:
    h = bar.get_height()
    ax0.annotate(f"{int(h):,}",
        xy=(bar.get_x() + bar.get_width() / 2, h),
        xytext=(0, 6), textcoords="offset points",
        ha="center", va="bottom", fontsize=10, fontweight="bold")
ax0.set_title("Distribución de clientes\nK-Means", fontweight="bold")
ax0.set_xlabel("Cluster")
ax0.set_ylabel("Número de clientes")

# Panel derecho: DBSCAN
ax1 = axes[1]
db_colors = [
    "#9CA3AF" if int(row["cluster"]) == -1
    else PALETTE_VIVID[i % len(PALETTE_VIVID)]
    for i, row in db_counts.iterrows()
]
bars1 = ax1.bar(db_counts["label"], db_counts["count"],
    color=db_colors, edgecolor="white", linewidth=0.8)
for bar in bars1:
    h = bar.get_height()
    ax1.annotate(f"{int(h):,}",
        xy=(bar.get_x() + bar.get_width() / 2, h),
        xytext=(0, 6), textcoords="offset points",
        ha="center", va="bottom", fontsize=10, fontweight="bold")

```

```

ax1.set_title("Distribución de clientes\nDBSCAN (gris = ruido)",
             fontweight="bold")
ax1.set_xlabel("Cluster / Ruido")
ax1.set_ylabel("Número de clientes")

plt.suptitle("K-Means vs DBSCAN - Distribución de clientes por cluster",
             fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig(
    FIGURES_CMP / "comparacion_distribucion_kmeans_dbscan.png",
    dpi=400, bbox_inches="tight", facecolor="white"
)
plt.show()
print("Gráfico guardado ")

```

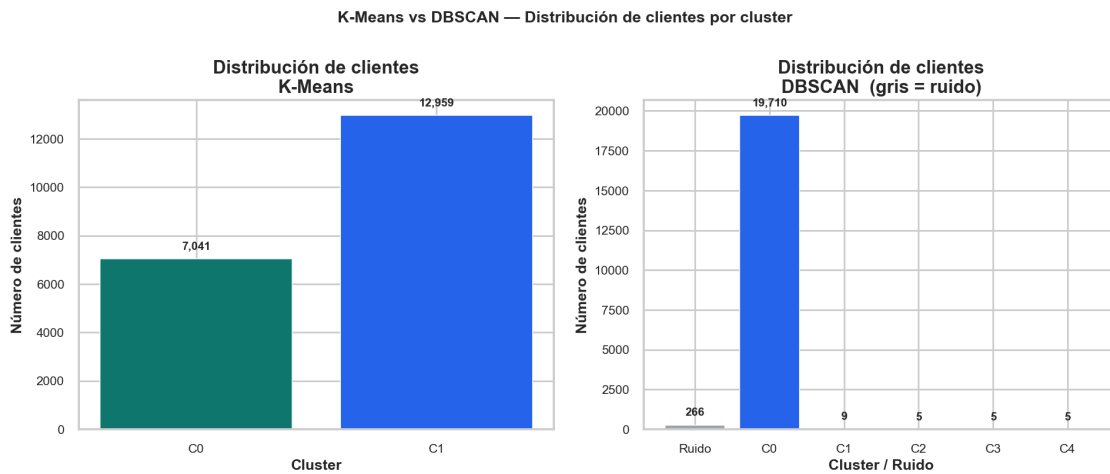


Gráfico guardado